ENTREGA DE MODELADO

# FloatMate Diagramas y mockup

# Gestión de flotas

Esta entrega presenta los cinco productos solicitados para un sistema con base de datos relacional: el modelo entidad relación, los casos de uso, las clases, los objetos y la primera parte del mockup de FloatMate.

## Tipo de base de datos

FloatMate utiliza PostgreSQL y un esquema relacional. Las nueve tablas de negocio se vinculan mediante claves primarias y foráneas. La distribución entre Terminal Norte y Terminal Sur cambia la ubicación de los registros, pero conserva el modelo relacional.

Los ejercicios adicionales con tipos compuestos y JSONB siguen ejecutándose en PostgreSQL. No se identificó una implementación de MongoDB ni de Neo4j en los archivos revisados.

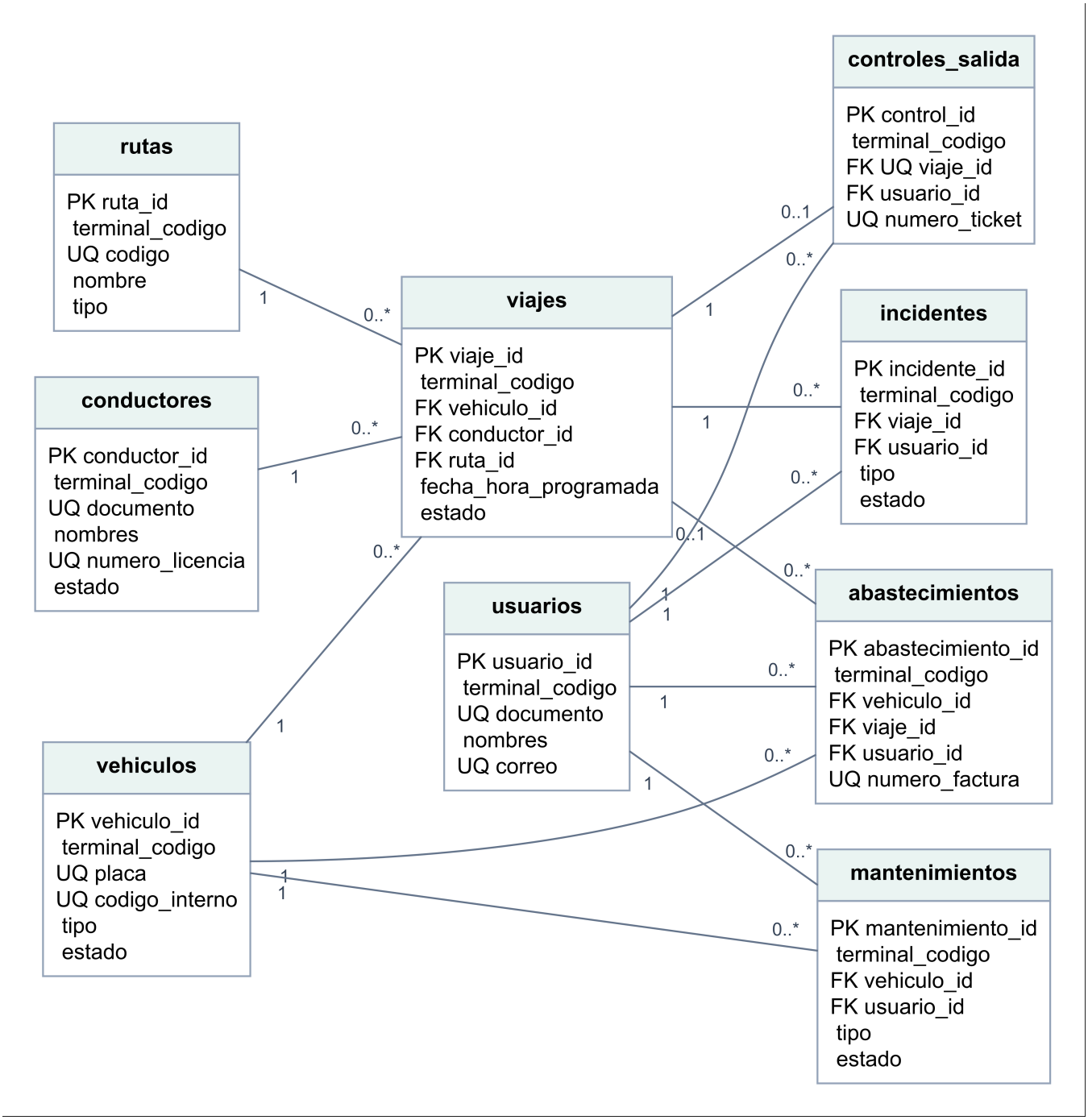
| Punto solicitado | Contenido entregado |
|---|---|
| 1 Diagrama entidad relación | 9 entidades, 12 relaciones, multiplicidades y diccionario de datos |
| 2 Diagrama de casos de uso | 3 actores, 15 casos de uso y especificación de los flujos iniciales |
| 3 Diagrama de clases | Modelo lógico del dominio con atributos, operaciones y asociaciones |
| 4 Diagrama de objetos | Ejemplo concreto con datos semilla del nodo Norte |
| 5 Parte 1 del mockup | 4 pantallas y 2 formularios navegables con validaciones |

Equipo: Tomás Yepes Arboleda, Cristian Benites y Mateo Gallo.
Fecha: 13 de octubre de 2026.

1 MODELO DE DATOS

# Diagrama entidad relación

Modelo lógico del esquema floatmate. Los atributos principales se muestran en el diagrama; el diccionario de las páginas siguientes contiene todas las columnas.




Notación: PK = clave primaria; FK = clave foránea; UQ = valor único. 1 = exactamente uno; 0..1 = cero o uno; 0..* = cero o muchos.

Cada viaje pertenece a un vehículo, un conductor y una ruta. Un viaje puede tener cero o un control de salida; un abastecimiento puede referirse a cero o un viaje. Cada registro operativo tiene un usuario responsable.

# Diccionario de entidades principales

Tipos y claves extraídos del DDL de Terminal Norte. El mismo modelo se replica en Sur. Todos los atributos son obligatorios salvo los señalados como NULL.

## usuarios

usuario_id bigint [PK]; terminal_codigo varchar(3); documento varchar(20) [UQ]; nombres varchar(60); apellidos varchar(60); correo varchar(120) [UQ]; rol varchar(20); activo boolean; creado_en timestamptz.

## conductores

conductor_id bigint [PK]; terminal_codigo varchar(3); documento varchar(20) [UQ]; nombres varchar(60); apellidos varchar(60); numero_licencia varchar(30) [UQ]; categoria_licencia varchar(4); licencia_vence date; telefono varchar(20); estado varchar(15).

## vehiculos

vehiculo_id bigint [PK]; terminal_codigo varchar(3); placa varchar(6) [UQ]; codigo_interno varchar(15) [UQ]; tipo varchar(25); marca varchar(40); modelo smallint; capacidad_pasajeros smallint; kilometraje numeric(12,2); proximo_mantenimiento_km numeric(12,2); estado varchar(20).

## rutas

ruta_id bigint [PK]; terminal_codigo varchar(3); codigo varchar(15) [UQ]; nombre varchar(100); origen varchar(100); destino varchar(100); tipo varchar(20); distancia_km numeric(8,2); duracion_minutos smallint; activa boolean.

Clave de fragmentación: terminal_codigo admite NOR en Norte y SUR en Sur. No existe una tabla terminales en el DDL, por lo que no se añade una entidad ficticia al diagrama.

# Diccionario de entidades operativas

Tipos y claves extraídos del DDL de Terminal Norte. El mismo modelo se replica en Sur. Todos los atributos son obligatorios salvo los señalados como NULL.

## viajes

viaje_id bigint [PK]; terminal_codigo varchar(3); vehiculo_id bigint [FK]; conductor_id bigint [FK]; ruta_id bigint [FK]; fecha_hora_programada timestamptz; fecha_hora_salida timestamptz [NULL]; fecha_hora_llegada timestamptz [NULL]; pasajeros smallint; estado varchar(15); observaciones varchar(250) [NULL].

## controles_salida

control_id bigint [PK]; terminal_codigo varchar(3); viaje_id bigint [FK, UQ]; usuario_id bigint [FK]; fecha_hora_control timestamptz; alcoholemia_mg_l numeric(5,3); resultado varchar(12); numero_ticket varchar(40) [UQ]; observaciones varchar(250) [NULL].

## abastecimientos

abastecimiento_id bigint [PK]; terminal_codigo varchar(3); vehiculo_id bigint [FK]; viaje_id bigint [FK] [NULL]; usuario_id bigint [FK]; fecha_hora timestamptz; litros numeric(8,2); valor_litro numeric(10,2); valor_total numeric(12,2); kilometraje numeric(12,2); estacion_servicio varchar(100); numero_factura varchar(40) [UQ].

## mantenimientos

mantenimiento_id bigint [PK]; terminal_codigo varchar(3); vehiculo_id bigint [FK]; usuario_id bigint [FK]; tipo varchar(15); fecha_ingreso date; fecha_salida date [NULL]; kilometraje numeric(12,2); costo numeric(12,2); proveedor varchar(100); descripcion varchar(300); estado varchar(15).

## incidentes

incidente_id bigint [PK]; terminal_codigo varchar(3); viaje_id bigint [FK]; usuario_id bigint [FK]; fecha_hora timestamptz; tipo varchar(25); severidad varchar(10); descripcion varchar(300); ubicacion varchar(150); tiene_evidencia boolean; estado varchar(15).

# Cardinalidades y reglas de integridad

| Relación | Desde el padre | Desde el registro hijo |
|---|---|---|
| vehiculos → viajes | 0..* | 1 |
| conductores → viajes | 0..* | 1 |
| rutas → viajes | 0..* | 1 |
| viajes → controles_salida | 0..1 | 1 |
| usuarios → controles_salida | 0..* | 1 |
| vehiculos → abastecimientos | 0..* | 1 |
| viajes → abastecimientos | 0..* | 0..1 |
| usuarios → abastecimientos | 0..* | 1 |
| vehiculos → mantenimientos | 0..* | 1 |
| usuarios → mantenimientos | 0..* | 1 |
| viajes → incidentes | 0..* | 1 |
| usuarios → incidentes | 0..* | 1 |

Reglas del DDL: placa con tres letras y tres dígitos; capacidad entre 10 y 80; origen diferente del destino; litros y precios positivos; total de combustible = redondeo(litros × valor_litro, 2).

Salida: APROBADO implica alcoholemia = 0; RECHAZADO implica alcoholemia > 0. La FK viaje_id de controles_salida es única, lo que impide dos controles para el mismo viaje.

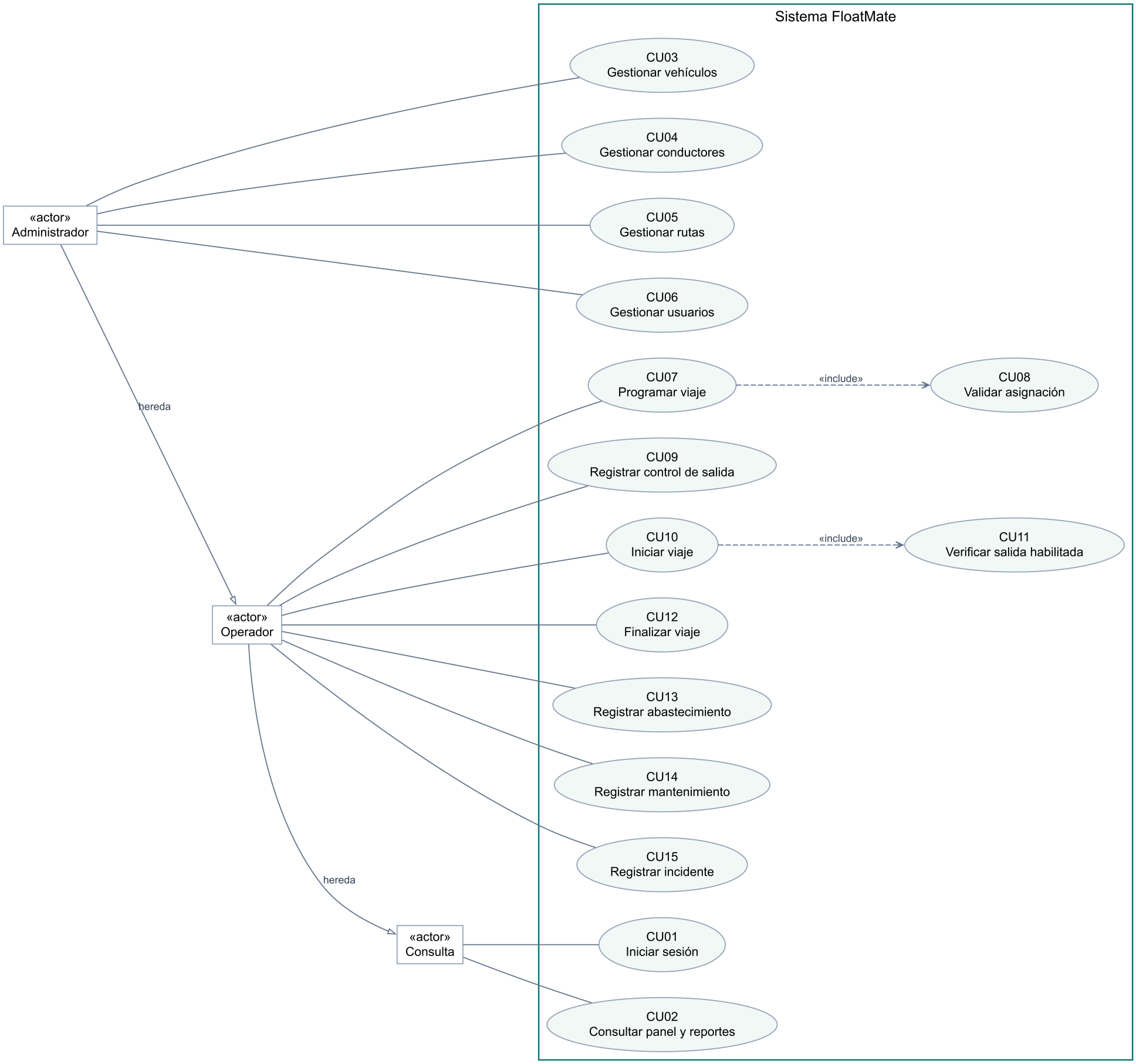
Programación: las parejas vehículo y hora, y conductor y hora, son únicas. Esto evita coincidencias en la misma hora programada; no garantiza por sí solo la ausencia de solapamientos durante toda la duración del recorrido.

Borrado: RESTRICT protege recursos usados; eliminar un viaje elimina su control por CASCADE. En abastecimientos, ON DELETE SET NULL permite conservar el registro si se elimina el viaje asociado.

Reglas programadas en Norte: el procedimiento de programación revisa disponibilidad, ruta activa, capacidad y hora; el trigger valida que los recursos pertenezcan a la misma terminal. Iniciar un viaje exige control aprobado y disponibilidad. Son controles diferentes de las FK.

# Diagrama de casos de uso

Diseño funcional del sistema. Los actores son roles de aplicación; se representan con la notación rectangular «actor» de UML. Los permisos se proponen como requisitos, no como un backend de autenticación existente.




Administrador especializa a Operador; Operador especializa a Consulta. Las asociaciones heredadas evitan repetir todas las líneas. Consulta accede al sistema y a los reportes; Operador registra la operación; Administrador también administra los catálogos y los usuarios.

«include»: programar un viaje incluye validar la asignación; iniciar un viaje incluye verificar que la salida esté habilitada. La flecha apunta al comportamiento incluido. No se usa «extend» para representar errores de validación.

El conductor es una entidad del negocio. No se dibuja como actor porque la parte inicial no contempla un acceso directo del conductor a FloatMate.

2 CASOS DE USO

# Catálogo y alcance funcional

| ID | Objetivo | Responsable | Mockup |
|---|---|---|---|
| CU01 | Iniciar sesión | Todos | Parte 1 |
| CU02 | Consultar panel y reportes | Consulta y herederos | Panel en parte 1 |
| CU03 | Gestionar vehículos | Administrador | Consultar y registrar |
| CU04 | Gestionar conductores | Administrador | Posterior |
| CU05 | Gestionar rutas | Administrador | Posterior |
| CU06 | Gestionar usuarios | Administrador | Posterior |
| CU07 | Programar viaje | Operador y Administrador | Parte 1 |
| CU08 | Validar asignación | Incluido por CU07 | Parte 1 |
| CU09 | Registrar control de salida | Operador y Administrador | Posterior |
| CU10 | Iniciar viaje | Operador y Administrador | Posterior |
| CU11 | Verificar salida habilitada | Incluido por CU10 | Posterior |
| CU12 | Finalizar viaje | Operador y Administrador | Posterior |
| CU13 | Registrar abastecimiento | Operador y Administrador | Posterior |
| CU14 | Registrar mantenimiento | Operador y Administrador | Posterior |
| CU15 | Registrar incidente | Operador y Administrador | Posterior |

La parte 1 ofrece credenciales de demostración y una sola sesión de Administrador. La política de permisos de los tres roles deberá implementarse en el servidor. Las pantallas restantes se reservan para siguientes partes.

# Especificación del flujo inicial

## CU01 Iniciar sesión

Actor: cualquier usuario. Precondición: usuario activo y credenciales válidas en el diseño final. Flujo: abrir acceso, escribir usuario y contraseña, enviar el formulario, verificar credenciales y mostrar el panel. Alternativa: credenciales incorrectas mantienen la pantalla y muestran un mensaje. Resultado: sesión habilitada. El prototipo simula este flujo con admin / floatmate123.

## CU03 Registrar vehículo

Actor: Administrador. Precondición: sesión activa y terminal seleccionada. Flujo: abrir Vehículos, elegir Registrar vehículo, completar placa, código, tipo, marca, modelo, capacidad, kilometraje, próximo mantenimiento y estado; validar y guardar. Alternativas: placa o código repetidos; formato inválido; números fuera del rango; campos vacíos. Resultado: vehículo disponible en la tabla y en los indicadores.

## CU07 Programar viaje y CU08 Validar asignación

Actor: Operador o Administrador. Precondiciones: sesión activa, ruta activa y recursos disponibles de la misma terminal. Flujo: abrir Viajes, elegir Programar viaje, seleccionar ruta, vehículo y conductor, indicar fecha, hora y pasajeros; validar disponibilidad, capacidad y ausencia de asignación a la misma hora; guardar y mostrar confirmación. Resultado: nuevo viaje con estado PROGRAMADO.

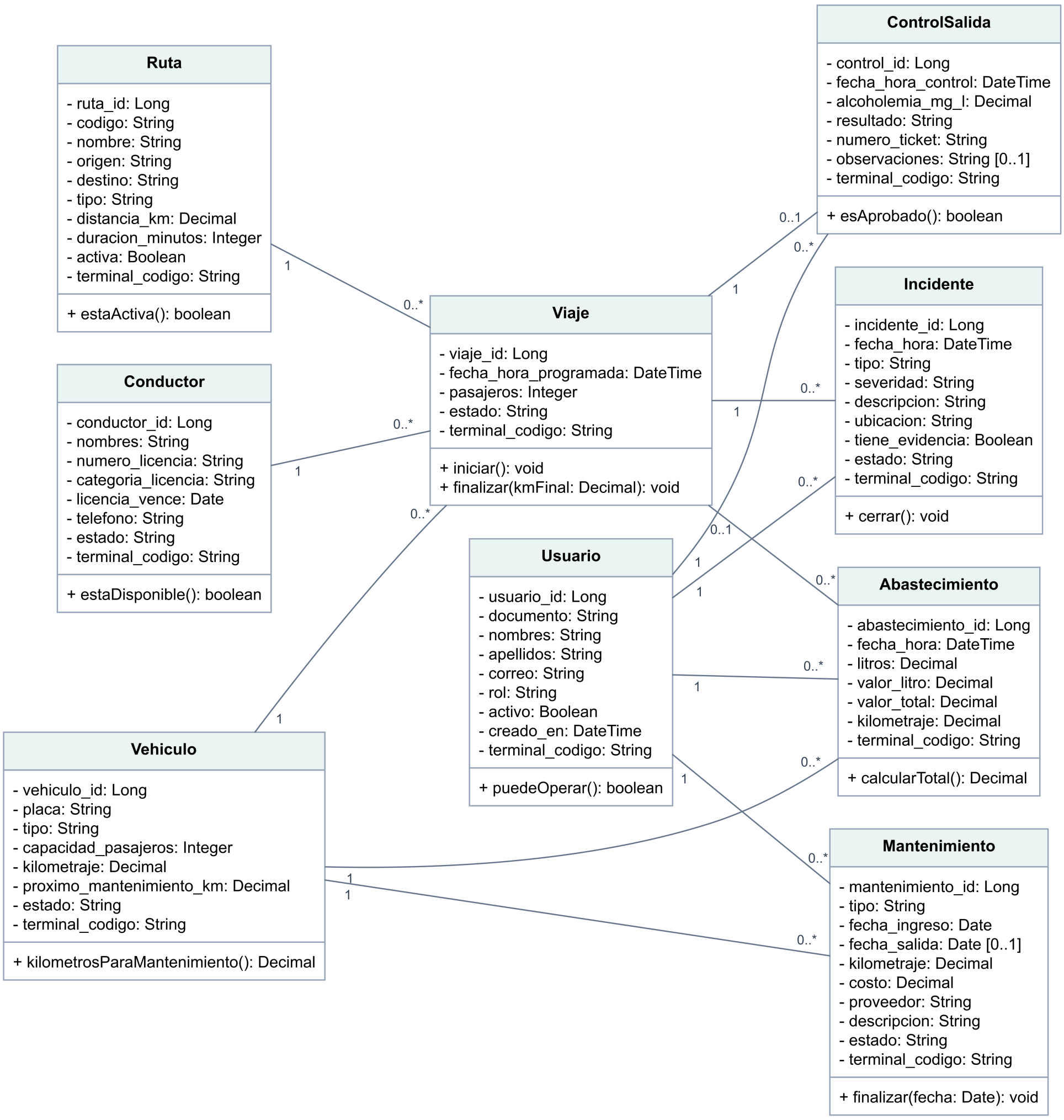
Alternativas: recurso sin disponibilidad, pasajeros superiores a la capacidad, horario ocupado o datos incompletos bloquean el guardado. El mockup también exige fecha futura como requisito propuesto de interfaz; esta condición no está declarada como CHECK en el DDL revisado.

## CU10 Iniciar viaje

El diseño exige viaje PROGRAMADO y verificación de salida habilitada: vehículo y conductor disponibles, y control APROBADO. Si falta el control o un recurso no está disponible, el inicio se rechaza. Esta función se modela para una parte posterior; no se ofrece un botón ficticio en la parte 1.

# Diagrama de clases del dominio

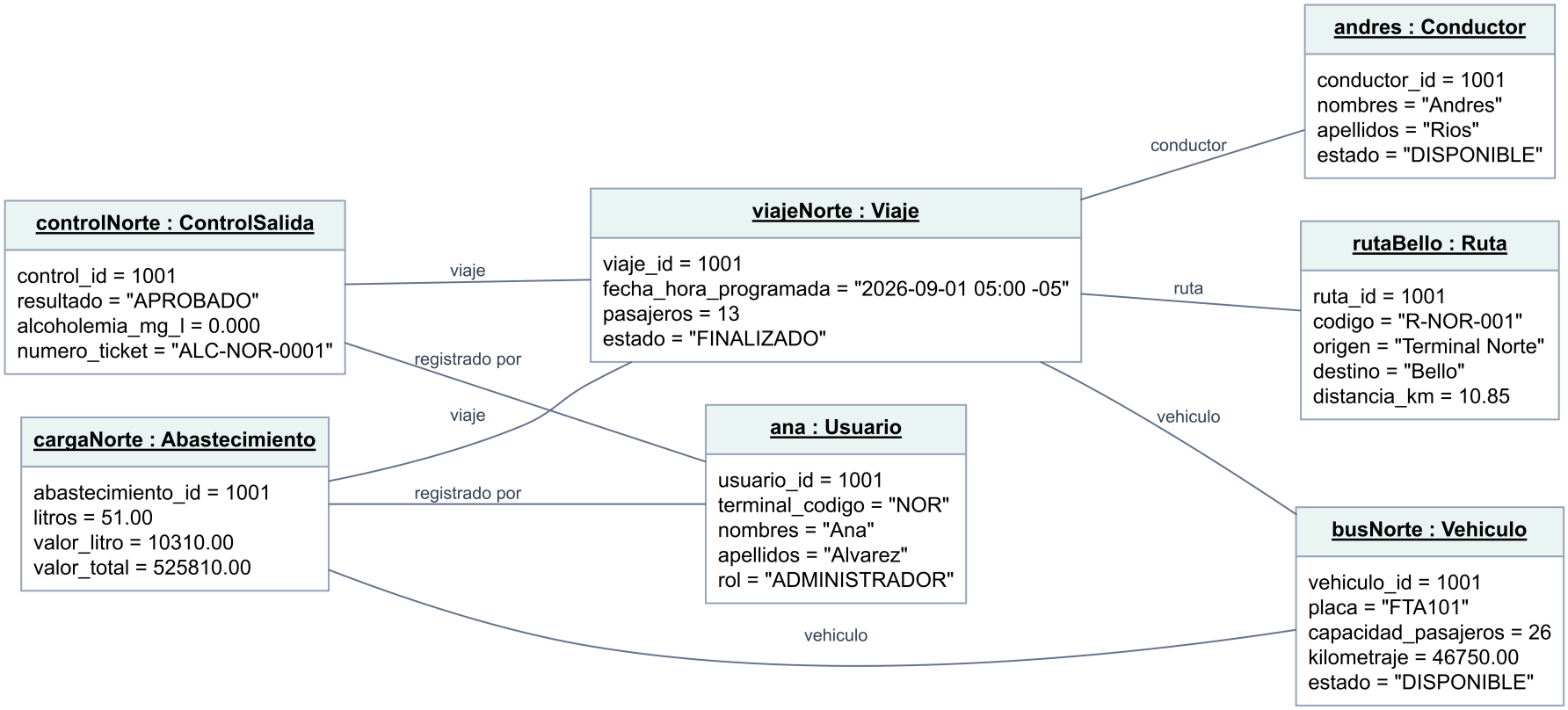
Modelo lógico propuesto para la aplicación completa. Las nueve clases corresponden a las entidades persistentes. No se presenta como copia literal del proyecto de consola del entregable de POO.


Ruta
- ruta_id: Long
- codigo: String
- nombre: String
- origen: String
- destino: String
- tipo: String
- distancia_km: Decimal
- duracion_minutos: Integer
- activa: Boolean
- terminal_codigo: String
+ estaActiva(): boolean
ControlSalida
- control_id: Long
- fecha_hora_control: DateTime
- alcoholemia_mg_l: Decimal
- resultado: String
- numero_ticket: String
- observaciones: String [0..1]
- terminal_codigo: String
+ esAprobado(): boolean
Viaje
- viaje_id: Long
- fecha_hora_programada: DateTime
- pasajeros: Integer
- estado: String
- terminal_codigo: String
+ iniciar(): void
+ finalizar(kmFinal: Decimal): void
Incidente
- incidente_id: Long
- fecha_hora: DateTime
- tipo: String
- severidad: String
- descripcion: String
- ubicacion: String
- tiene_evidencia: Boolean
- estado: String
- terminal_codigo: String
+ cerrar(): void
Conductor
- conductor_id: Long
- nombres: String
- numero_licencia: String
- categoria_licencia: String
- licencia_vence: Date
- telefono: String
- estado: String
- terminal_codigo: String
+ estaDisponible(): boolean
Usuario
- usuario_id: Long
- documento: String
- nombres: String
- apellidos: String
- correo: String
- rol: String
- activo: Boolean
- creado_en: DateTime
- terminal_codigo: String
+ puedeOperar(): boolean
Abastecimiento
- abastecimiento_id: Long
- fecha_hora: DateTime
- litros: Decimal
- valor_litro: Decimal
- valor_total: Decimal
- kilometraje: Decimal
- terminal_codigo: String
+ calcularTotal(): Decimal
Vehiculo
- vehiculo_id: Long
- placa: String
- tipo: String
- capacidad_pasajeros: Integer
- kilometraje: Decimal
- proximo_mantenimiento_km: Decimal
- estado: String
- terminal_codigo: String
+ kilometrosParaMantenimiento(): Decimal
Mantenimiento
- mantenimiento_id: Long
- tipo: String
- fecha_ingreso: Date
- fecha_salida: Date [0..1]
- kilometraje: Decimal
- costo: Decimal
- proveedor: String
- descripcion: String
- estado: String
- terminal_codigo: String
+ finalizar(fecha: Date): void
1
0..*
0..1
1
0..*
1
0..*
1
0..*

4 INSTANCIAS CONCRETAS

# Diagrama de objetos

Escenario extraído de los datos semilla de Terminal Norte. Representa siete objetos existentes en un mismo ejemplo; los nombres de instancia y clase se subrayan conforme a UML.




viajeNorte enlaza busNorte, andres y rutaBello. controlNorte y cargaNorte se vinculan al viaje y al usuario ana. Los enlaces son instancias de las asociaciones del diagrama de clases.

Comprobación: 13 pasajeros ≤ 26 puestos; 51 litros × 10 310 = 525 810; control APROBADO y alcoholemia 0. El viaje está FINALIZADO y sus recursos pueden volver a estar DISPONIBLES. Todos los objetos del ejemplo pertenecen a NOR.

5 PARTE 1 DEL MOCKUP

# Alcance y navegación del mockup

La primera parte muestra el recorrido operativo inicial en un prototipo de alta fidelidad. Se reutiliza el propósito del proyecto web existente, con formularios ajustados a los campos de PostgreSQL y catálogos seleccionables para evitar escribir referencias como texto libre.

| Pantalla | Acción principal | Caso de uso |
|---|---|---|
| Acceso | Ingresar y visualizar error de credenciales | CU01 |
| Resumen | Consultar inventario, disponibilidad y viajes | CU02 |
| Vehículos | Buscar, filtrar y abrir registro | CU03 |
| Registrar vehículo | Validar y guardar un vehículo | CU03 |
| Viajes | Consultar y filtrar programación | CU02 / CU07 |
| Programar viaje | Seleccionar recursos, validar y guardar | CU07 / CU08 |

## Recorrido de navegación

Acceso → Resumen → Vehículos → Registrar vehículo → Vehículos.
Resumen o Viajes → Programar viaje → confirmación → listado actualizado.
Cerrar sesión → Acceso.

## Comportamiento verificable

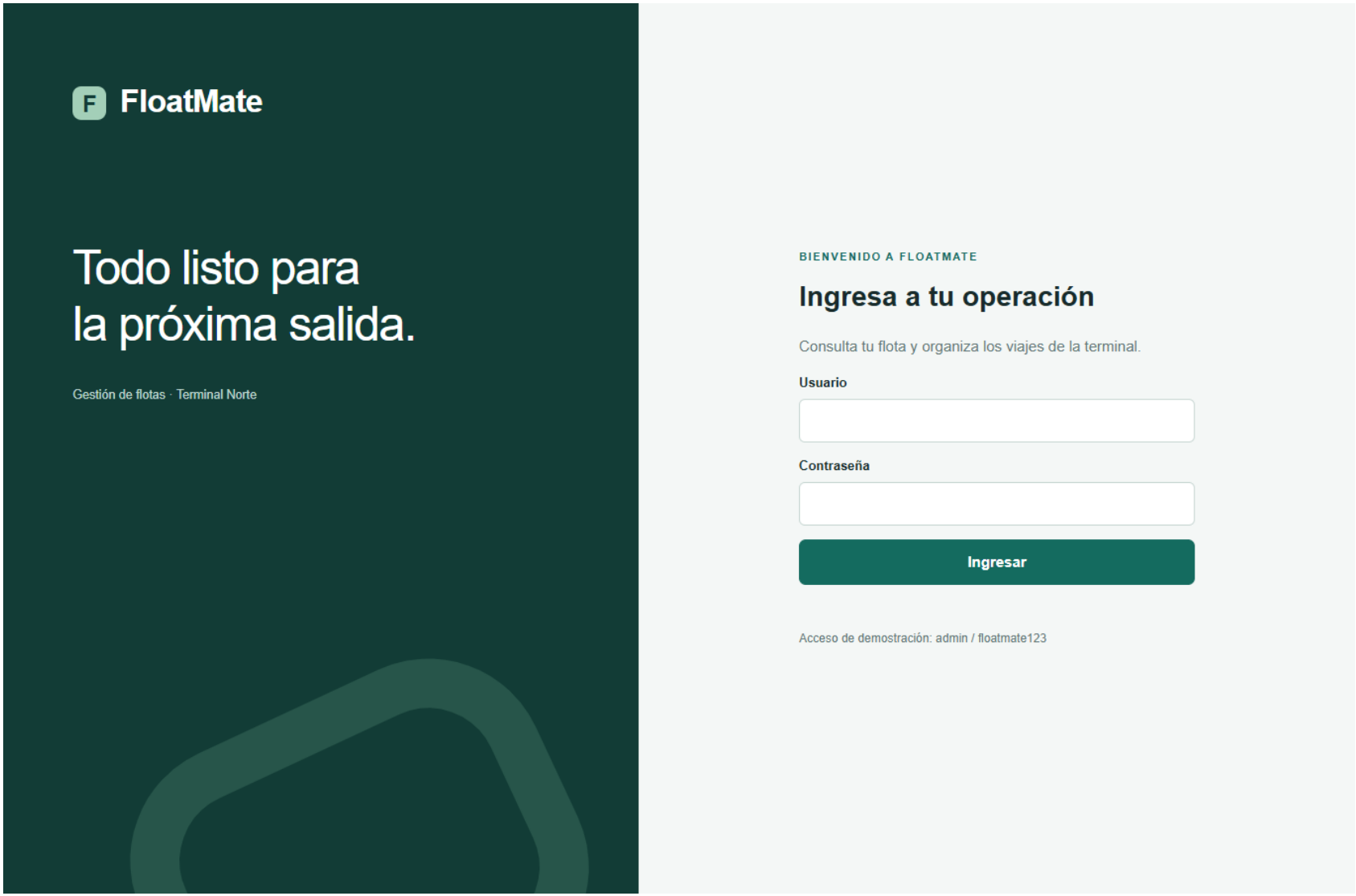
Los botones abren formularios, los filtros actualizan las tablas y los registros se reflejan en los indicadores. La interfaz muestra errores junto al formulario y mensajes de confirmación. Los diálogos tienen cancelación, navegación con teclado y campos etiquetados; las tablas permiten desplazamiento horizontal en pantallas pequeñas.

## Límites de esta parte

La sesión es demostrativa. Los datos se guardan en el navegador; no se leen ni escriben en PostgreSQL. Rutas y conductores son catálogos de muestra de NOR. Los viajes futuros son ejemplos de interfaz; no forman parte del escenario histórico del diagrama de objetos. Las etapas de salida, combustible, mantenimiento e incidentes se desarrollan después.

5 EVIDENCIA DEL MOCKUP

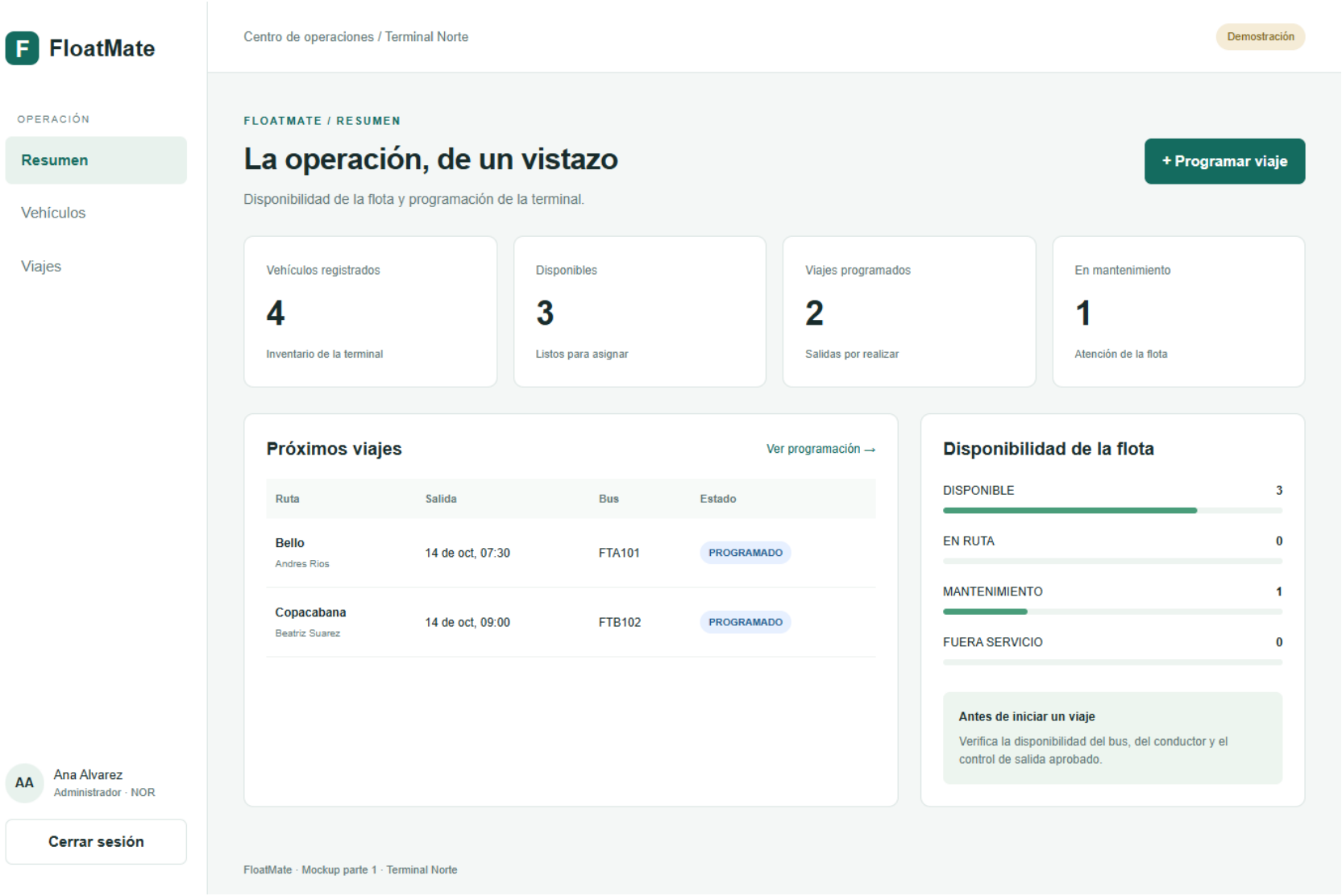
# Acceso al sistema




Usuario y contraseña con validación y acceso de demostración. Los errores se muestran sin abandonar la pantalla.

5 EVIDENCIA DEL MOCKUP

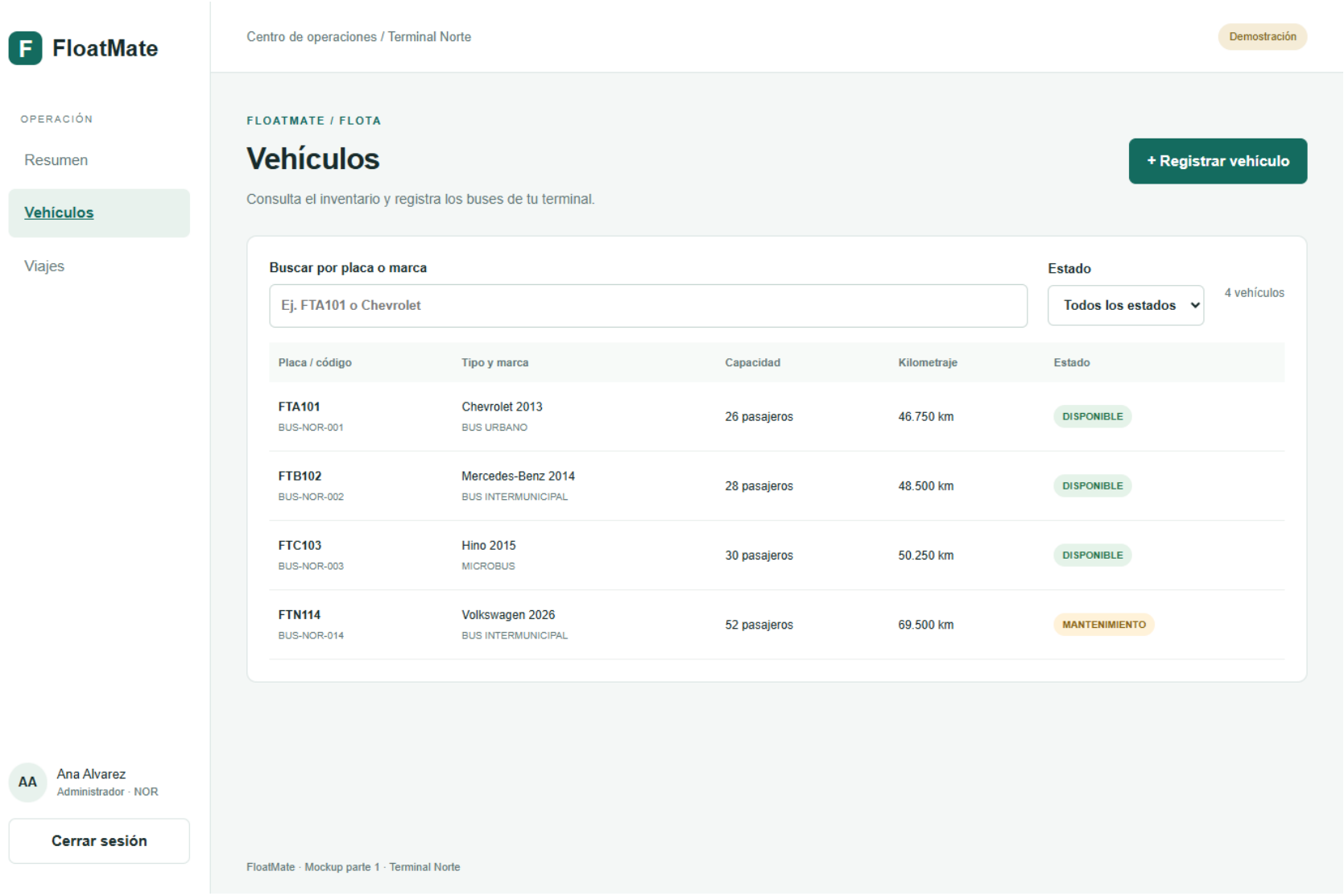
# Panel principal




Indicadores calculados a partir de los registros del prototipo. Navegación hacia vehículos, viajes y programación de una salida.

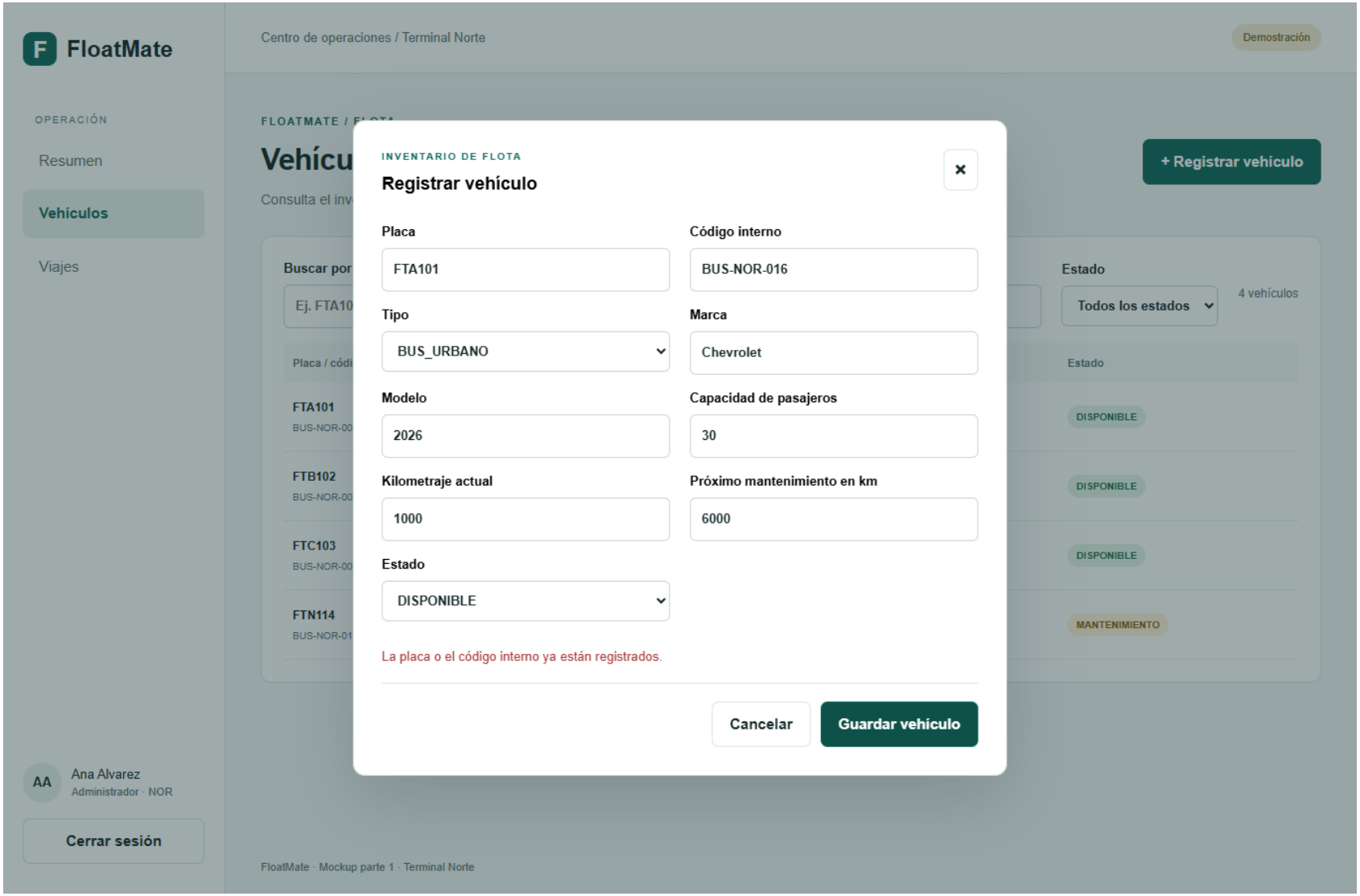
5 EVIDENCIA DEL MOCKUP

# Consulta de vehículos




Búsqueda por placa o marca, filtro de estado y tabla con información del inventario. Los estados usan los valores del modelo relacional.

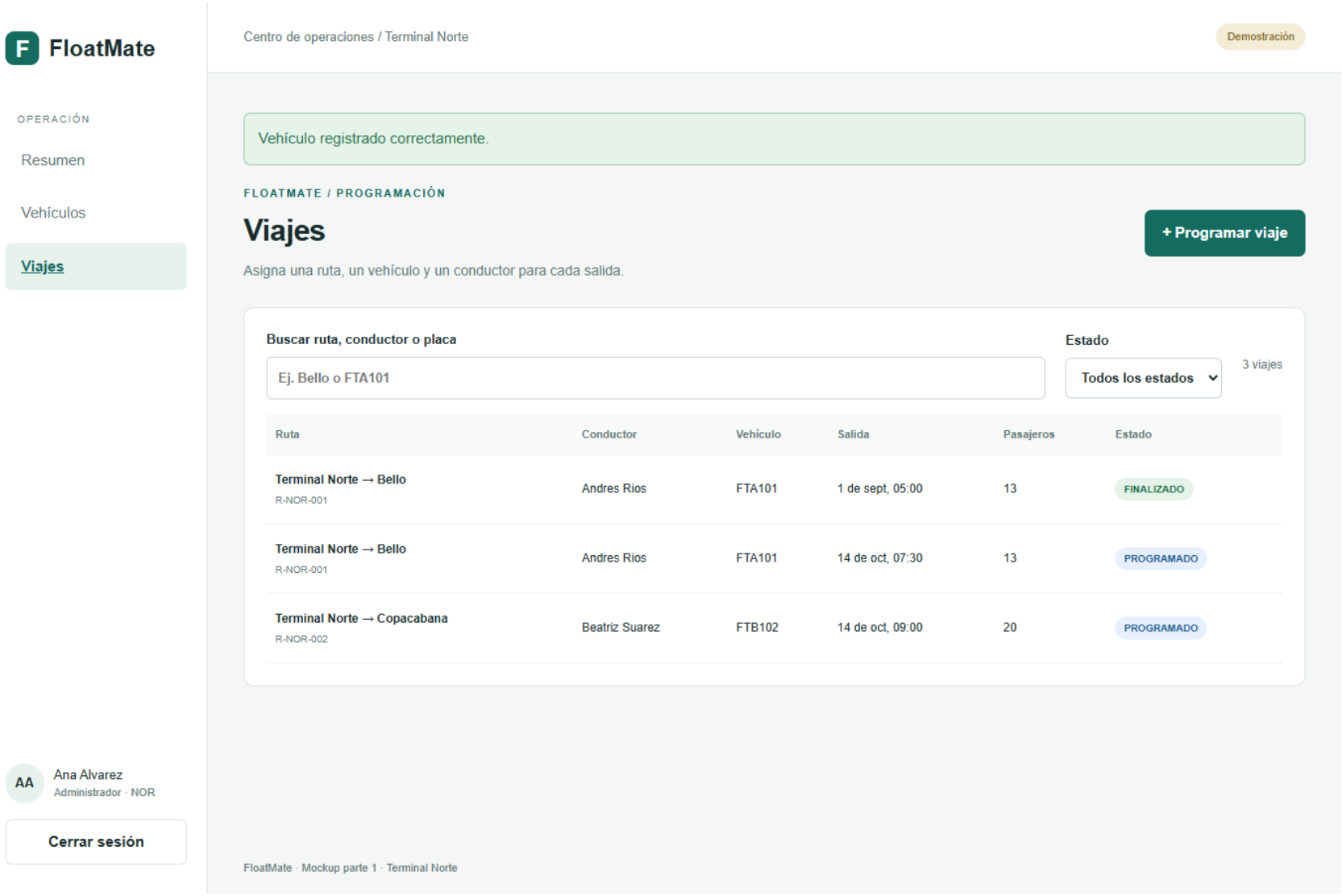
# Formulario de vehículo




Campos alineados con vehiculos. El ejemplo muestra un intento de registrar una placa existente y el mensaje que bloquea el guardado.

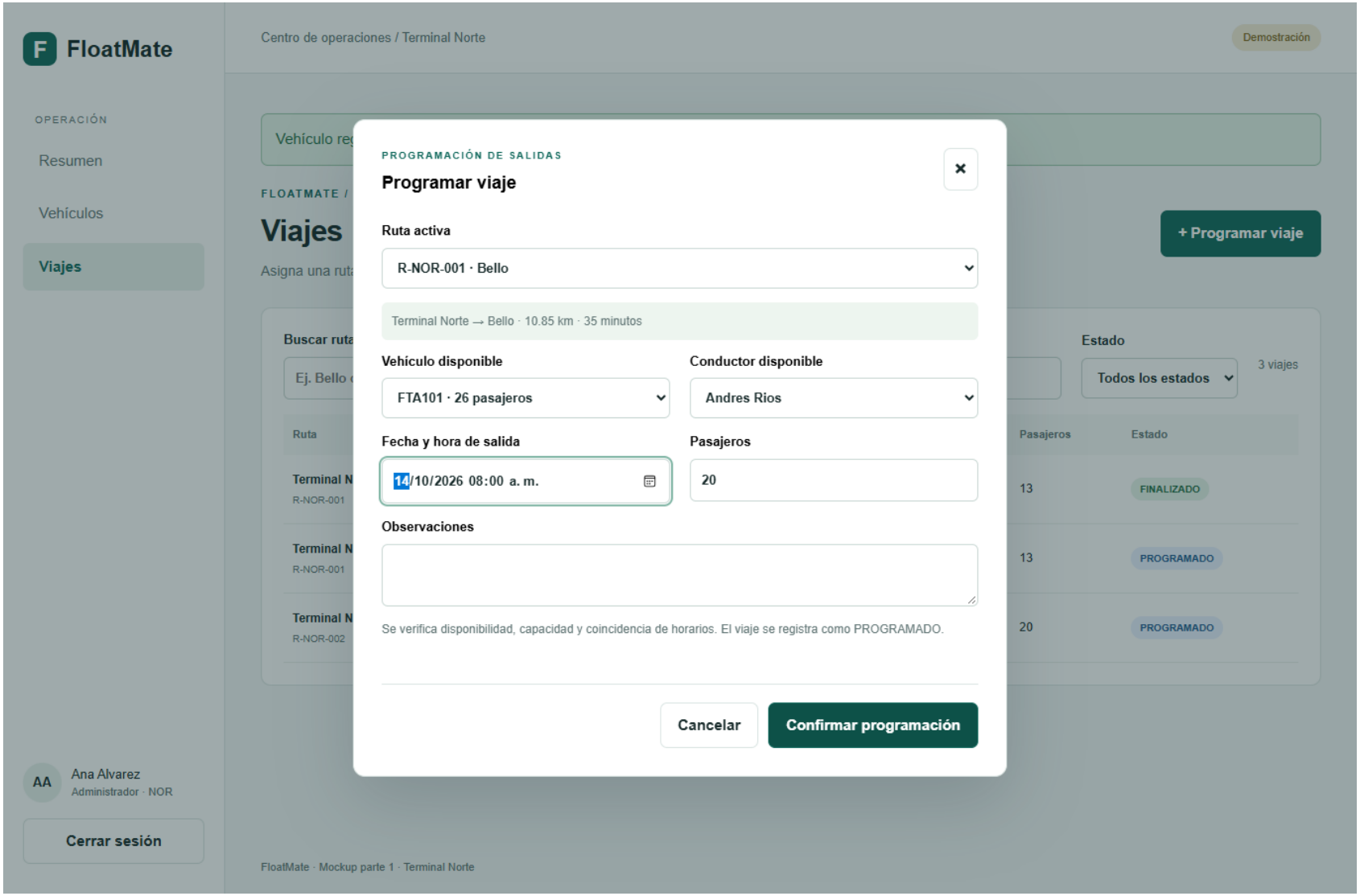
5 EVIDENCIA DEL MOCKUP

# Consulta de viajes




Ruta, conductor, vehículo, salida, pasajeros y estado. El listado se actualiza al confirmar una nueva programación.

# Formulario de programación




Ruta, vehículo y conductor se seleccionan por catálogo. Se validan capacidad, disponibilidad y asignación a la misma hora antes de confirmar.

VERIFICACIÓN DE LA ENTREGA

# Trazabilidad y sustentación

| Requisito | Archivo o evidencia | Verificación |
|---|---|---|
| Entidad relación | diagramas/01_entidad_relacion.svg | 9 entidades y 12 FK del DDL |
| Casos de uso | diagramas/02_casos_de_uso.svg | 3 actores; 15 objetivos; include |
| Clases | diagramas/03_clases.svg | Atributos, operaciones y multiplicidades |
| Objetos | diagramas/04_objetos.svg | Instancias y enlaces coherentes |
| Mockup parte 1 | mockup/index.html | 4 pantallas y 2 formularios |

## Qué explicar durante la sustentación

1. FloatMate es relacional porque organiza los datos en tablas con PK y FK.
2. Viaje conecta un bus, un conductor y una ruta; el control es opcional antes de registrar la salida.
3. Las multiplicidades UML conservan las reglas del esquema.
4. Las clases describen tipos de objetos; el diagrama de objetos muestra valores concretos.
5. El mockup permite demostrar el flujo inicial; la conexión real a PostgreSQL corresponde a la implementación posterior.

## Validación realizada

Se contrastaron las nueve entidades y las doce FK con el SQL. En el navegador se verificaron acceso válido e inválido, filtros vacíos, placa duplicada, creación de vehículo, capacidad excedida, fecha pasada, programación de viaje, conflicto de horario, persistencia de datos, vista móvil sin desbordamiento y cierre de sesión. No se ejecutaron pruebas contra un servidor PostgreSQL.

## Fuentes del modelo

Proyecto: database/01_nodo_norte/01_modelo_norte.sql; database/01_nodo_norte/02_datos_norte.sql; database/05_programacion_bd/01_todo_funciones_procedimientos_triggers.sql. Alcance web: README del proyecto FloatMate de Programación Web. Modelo completo disponible en fuentes/modelo.json.

Referencia de notación: OMG UML 2.5.1. Referencia de integridad: PostgreSQL restricciones.